

Khmelnyskyi National University
Department of Information Technonogy

Report

Laboratory work №2

Discipline: “Programming”

Topic: “*Branched algorithms*”

Completed: 1st year student, group SE-26-1

Nykon Hrytsyshyn
Name, Surname

Checked:

Valeriy Martynyuk
Name, Surname

Purpose: To to acquire skills in the use of conditional operators in the program and the compilation of block diagrams of programs and programs in the C language.

Technical Environment: The project is implemented in accordance with the C11 standard and configured for cross-platform compilation using CMake and the Ninja build system. The application is built and verified using modern C compilers (GCC, Clang, and MSVC) with strict diagnostic flags enabled, ensuring high code quality, portability, and compatibility across operating systems.

Note: The software implementation is designed using a modular architecture with robust input validation and an interactive task execution engine. Detailed flowcharts illustrating the overall execution logic, along with full source code for all program components (tasks.h, utils.h, main.c, runner.c, and utils.c), are provided in Appendices A...F.

Task 1

Write a program that solves the task according to the option.

1) Given two real numbers x and y . Print the number of the quadrant on the coordinate plane to which the point (x, y) belongs.

Algorithm Description & Mathematical Formulation:

A point (x, y) in a 2D Cartesian coordinate system is classified based on the signs of its coordinates:

- Quadrant I: $x > 0$ and $y > 0$;
- Quadrant II: $x < 0$ and $y > 0$;
- Quadrant III: $x < 0$ and $y < 0$;
- Quadrant IV: $x > 0$ and $y < 0$;

Boundary cases (when $x = 0$ or $y = 0$) are processed prior to quadrant determination to handle points lying on the coordinate axes or at the origin $(0,0)$ correctly. Input data validity is checked using standard I/O helper functions.

					LW-SE-26-1 09			
Ch.	Sheet	Nº document	Signature	Date				
Developed		Hrytsyshyn N.B.			Laboratory work Nº2			
Checked		Martynyuk V.V.						
N.control								
Confirmed								
						Letter	Sheet	Sheets
						H	1	23
						KNU group SE-26-1		

1.1. Block diagram of the program algorithm

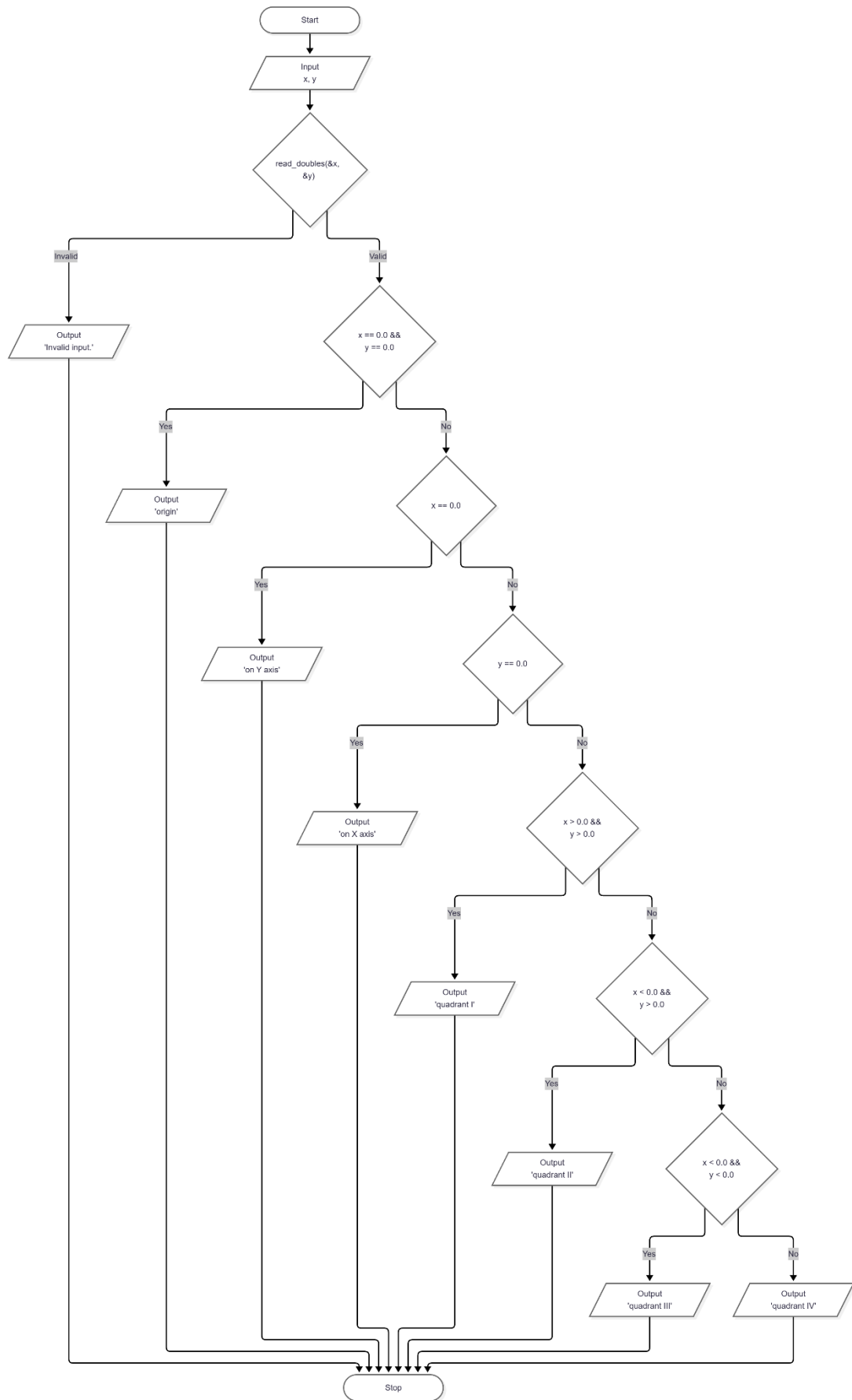


Fig. 1 – Block diagram of the program algorithm

1.2. Program text

```
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

static const char *get_point_location(const double x, const double y) {
    const int x_is_zero = x == 0.0;
    const int y_is_zero = y == 0.0;

    if (x_is_zero && y_is_zero) {
        return "The point is the origin.";
    }

    if (x_is_zero) {
        return "The point lies on the Y axis.";
    }

    if (y_is_zero) {
        return "The point lies on the X axis.";
    }

    return x > 0.0 ? (y > 0.0 ? "The point belongs to quadrant I."
                               : "The point belongs to quadrant IV.")
                 : (y > 0.0 ? "The point belongs to quadrant II."
                               : "The point belongs to quadrant III.");
}

void run_task1(void) {
    double x = 0.0;
    double y = 0.0;

    puts("\n=== Task 1: quadrant of a point (x, y) ===\n");

    if (!read_doubles("Enter x, y: ", 2, &x, &y)) {
        puts("Invalid input.");
        return;
    }

    printf("\nPoint: (%.4f, %.4f)\n", x, y);
    puts(get_point_location(x, y));
}
```

1.3. Program results

```
=== Task 1: quadrant of a point (x, y) ===  
Enter x, y: -1 1  
Point: (-1.0000, 1.0000)  
The point belongs to quadrant II.
```

Fig. 2 – Task 1 results

Task 2

Write a program according to the option:

4) The given age of a person. Write a program that analyzes a person according to his age and assigns him to one of four groups: preschooler, schoolboy, worker, pensioner.

Implementation Details:

The age classification module is designed around a clean separation of concerns, decoupling data validation, core domain logic, and user interface formatting.

Input processing relies on the custom “read_long()” utility function, which guarantees robust parsing and gracefully rejects non-numeric or malformed user input. Once parsed, integer values are validated against an operational age threshold of [0, 130] years; any input exceeding these reasonable limits triggers explicit error handling without executing the classification pipeline.

The core logic in ‘age.h’ leverages a custom enumeration (“AgeGroup”), dedicated conversion functions (“classify_age()”, “age_group_to_str()”), and a lookup array (“AGE_RULES”) to evaluate the age sequentially across four non-overlapping brackets: Preschooler (0...6 years), Schoolboy (7...17 years), Worker (18...59 years), and Pensioner (60+ years).

Decoupling rule evaluation from string formatting ensures high code maintainability, extensibility, and seamless integration with the task execution engine in “runner.c”.

2.1. Block diagram of the program algorithm

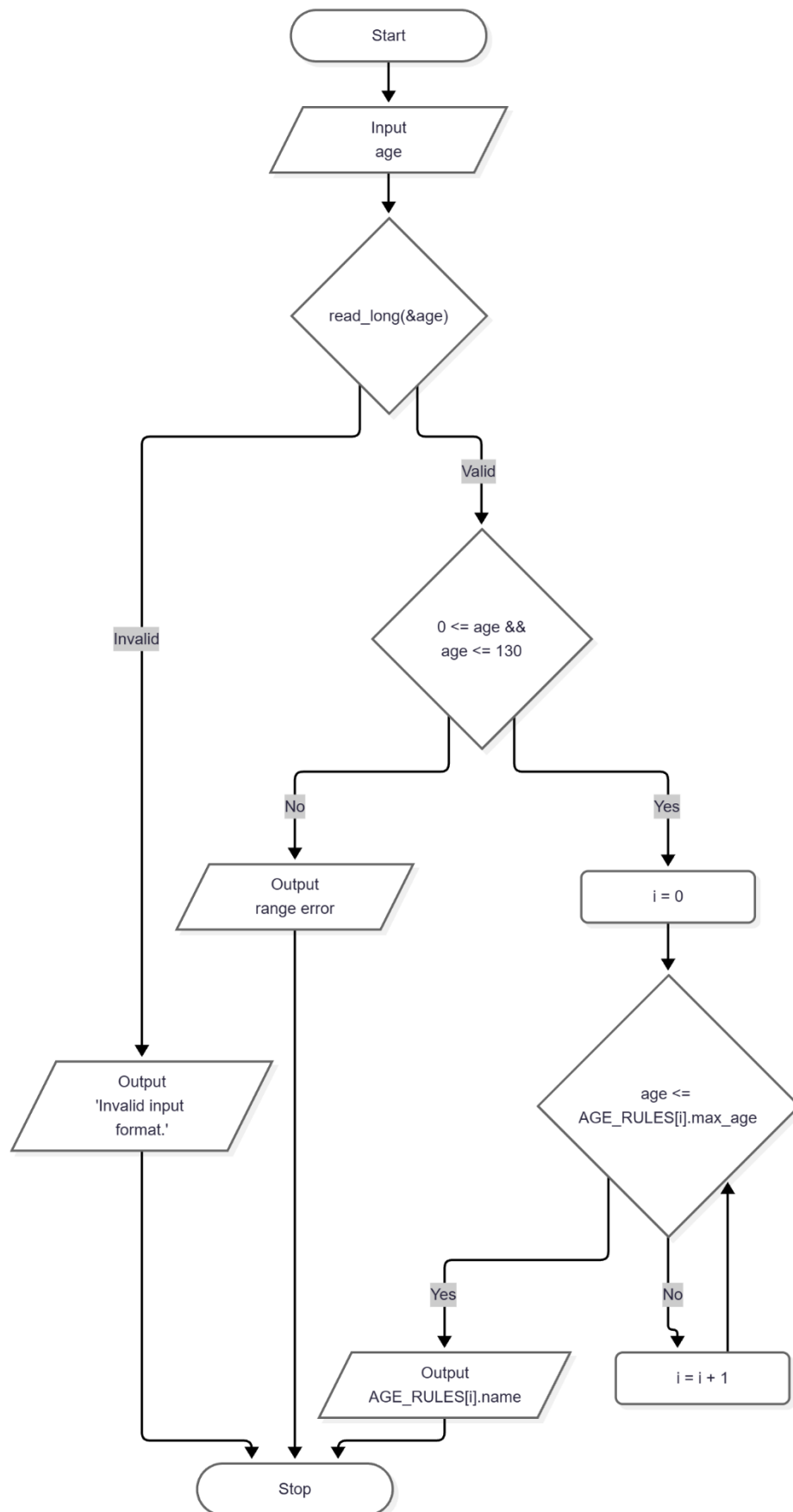


Fig. 3 – Block diagram of the program algorithm

2.2. Program text

```
#ifndef AGE_H
#define AGE_H
#include <stddef.h>

typedef enum {
    AGE_GROUP_INVALID = -1,
    AGE_GROUP_PRECHOOLER,
    AGE_GROUP_SCHOOLBOY,
    AGE_GROUP_WORKER,
    AGE_GROUP_PENSIONER
} AgeGroup;

typedef struct {
    long max_age;
    AgeGroup group;
    const char *name;
} AgeGroupRule;

enum { MIN_PLAUSIBLE_AGE = 0, MAX_PLAUSIBLE_AGE = 130 };

const AgeGroupRule AGE_RULES[] = {
    {.max_age = 6, .group = AGE_GROUP_PRECHOOLER, .name = "preschooler"},
    {.max_age = 17, .group = AGE_GROUP_SCHOOLBOY, .name = "schoolboy"},
    {.max_age = 59, .group = AGE_GROUP_WORKER, .name = "worker"},
    {.max_age = 130, .group = AGE_GROUP_PENSIONER, .name = "pensioner"},
};

const size_t RULES_COUNT = sizeof(AGE_RULES) / sizeof(AGE_RULES[0]);

AgeGroup classify_age(long age);
const char *age_group_to_str(AgeGroup group);

#endif // AGE_H

#include "age.h"
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

AgeGroup classify_age(const long age) {
    if (age < MIN_PLAUSIBLE_AGE || age > MAX_PLAUSIBLE_AGE) {
        return AGE_GROUP_INVALID;
    }
}
```

```

    }

    for (size_t i = 0; i < RULES_COUNT; ++i) {
        const AgeGroupRule rule = AGE_RULES[i];

        if (age <= rule.max_age) {
            return rule.group;
        }
    }

    return AGE_GROUP_INVALID;
}

const char *age_group_to_str(const AgeGroup group) {
    for (size_t i = 0; i < RULES_COUNT; ++i) {
        const AgeGroupRule rule = AGE_RULES[i];

        if (group == rule.group) {
            return rule.name;
        }
    }

    return "invalid";
}

void run_task2(void) {
    long age = 0;

    puts("\n=== Task 2: age group classification ===\n");

    if (!read_long("Enter age (years): ", &age)) {
        puts("Invalid input.");
        return;
    }

    const AgeGroup group = classify_age(age);

    if (group == AGE_GROUP_INVALID) {
        printf(
            "Error: age must be between %d and %d.\n",
            MIN_PLAUSIBLE_AGE,
            MAX_PLAUSIBLE_AGE
        );
        return;
    }
}

```

```

printf("\nAge: %ld\n", age);
printf("Group: %s.\n", age_group_to_str(group));
}

```

2.3. Program results

```

=== Task 2: age group classification ===

Enter age (years): 12

Age: 12
Group: schoolboy.

```

Fig. 4 – Task 2 results

Task 3

The function $y = f(x)$ is given graphically, calculate the value of this function (set x independently).

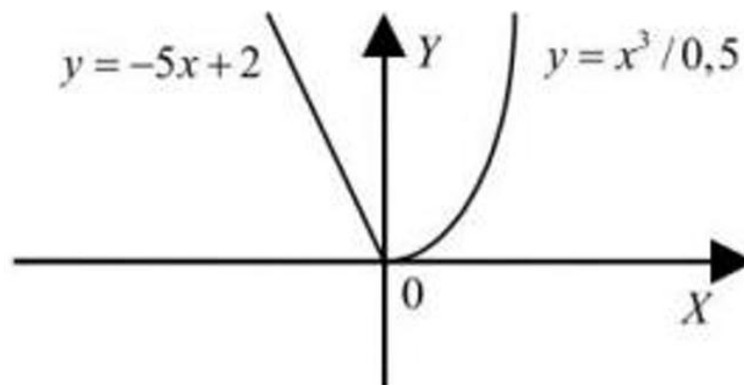


Fig. 5 – The graph of functions

The piecewise function evaluation module computes $y = f(x)$ based on the graphical definition, accommodating a jump discontinuity at $x = 0$. Input acquisition is handled via the helper function “read_double()”, which safely parses double-precision floating-point numbers from standard input while intercepting non-numeric tokens and buffer errors.

The computational logic is partitioned into two distinct analytical branches: for negative arguments ($x < 0$), the function evaluates the linear relation $y = -5x + 2$; for non-negative values ($x \geq 0$), it calculates the cubic curve $y = x^3 / 0.5$.

3.1. Block diagram of the program algorithm

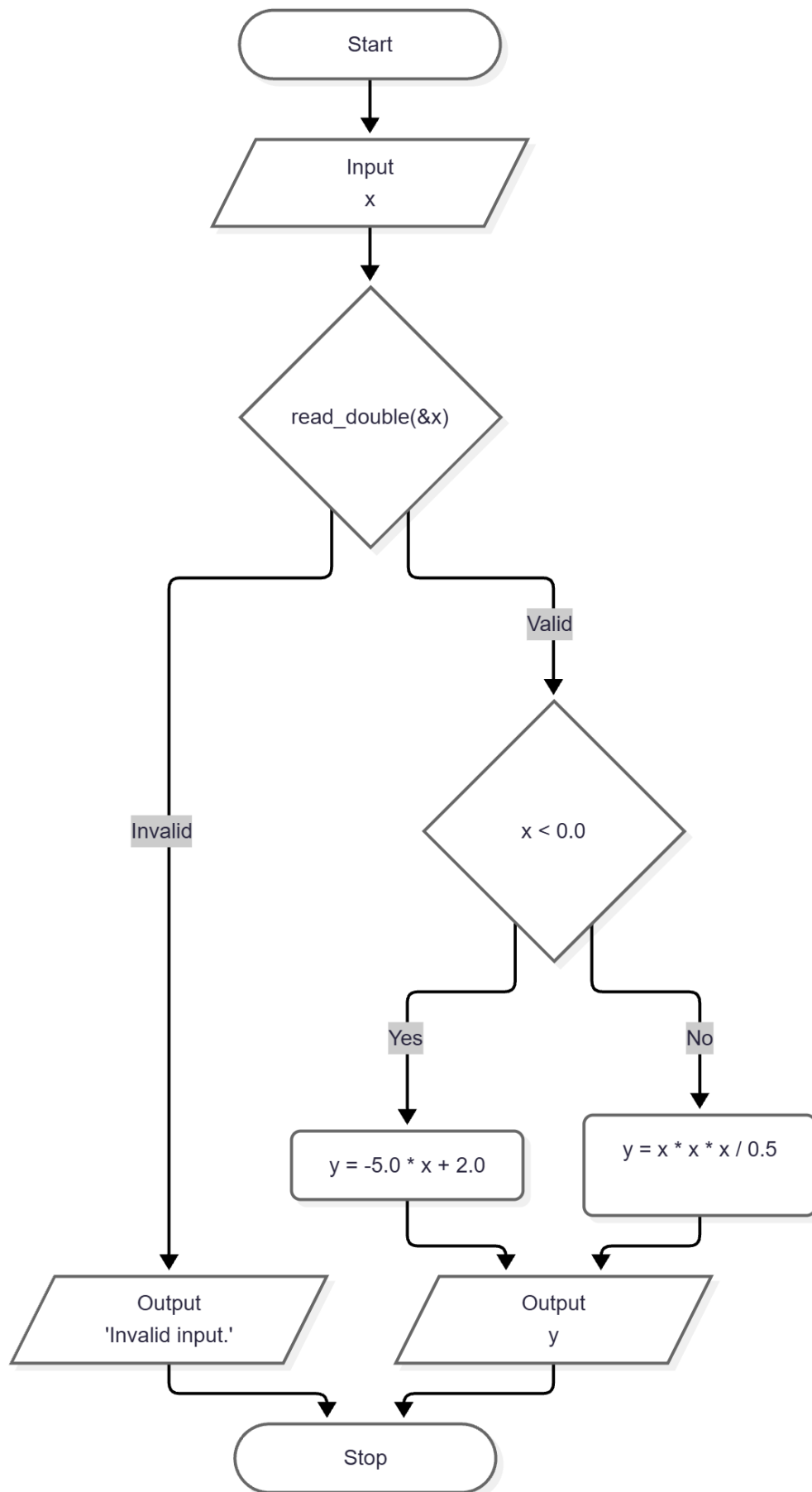


Fig. 6 – Block diagram of the program algorithm

3.2. Program text

```
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

typedef struct {
    double value;
    const char *formula;
} Result;

static Result calculate_pieewise(const double x) {
    return x < 0.0
        ? (Result){.value = -5.0 * x + 2.0, .formula = "y = -5x + 2"}
        : (Result){.value = 2.0 * x * x * x, .formula = "y = x^3 / 0.5"};
}

void run_task3(void) {
    double x = 0.0;

    puts("\n=== Task 3: pieewise function ===\n");
    puts("Formula: y = -5x + 2 for x < 0, y = x^3 / 0.5 for x >= 0");

    if (!read_double("Enter x: ", &x)) {
        puts("Invalid input.");
        return;
    }

    const Result result = calculate_pieewise(x);

    printf("\nx = %.4f\n", x);
    printf("Branch used: %s\n", result.formula);
    printf("y = %.4f\n", result.value);
}
```

3.3. Program results

```
=== Task 3: pieewise function ===

Formula: y = -5x + 2 for x < 0, y = x^3 / 0.5 for x >= 0
Enter x: 13

x = 13.0000
Branch used: y = x^3 / 0.5
y = 4394.0000
```

Fig. 7 – Task 3 results

Task 4

Write a program that solves the task. Task option:

- 4) Given a positive four-digit integer N. Determine the sum the first three digits.

4.1. Block diagram of the program algorithm

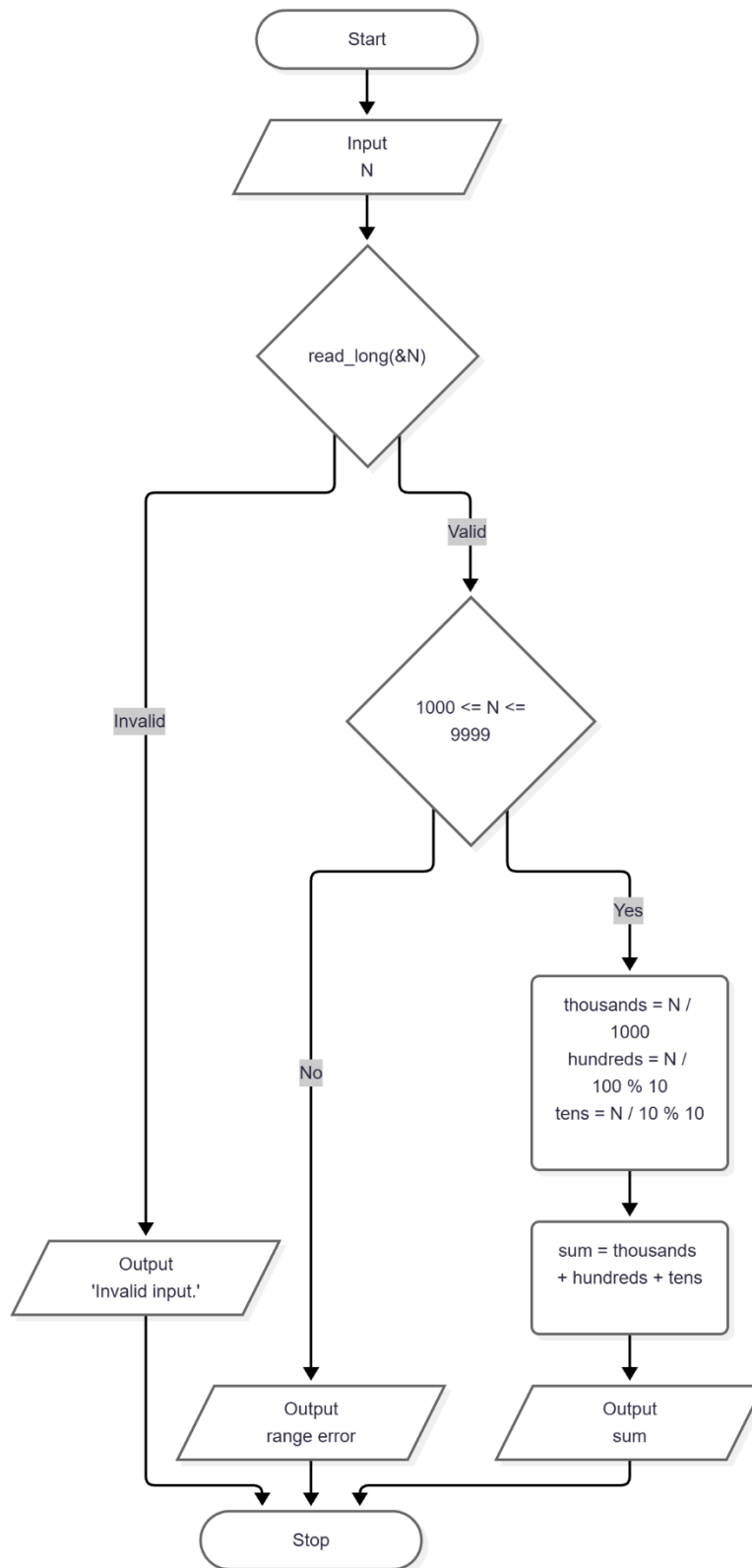


Fig. 8 – Block diagram of the program algorithm

4.2. Program text

```
#ifndef DIGITS_H
#define DIGITS_H

enum { MIN_FOUR_DIGIT = 1000, MAX_FOUR_DIGIT = 9999 };

typedef struct {
    int thousands;
    int hundreds;
    int tens;
    int units;
} Digits4;

Digits4 extract_digits4(int num);

#endif // DIGITS_H

#include "digits.h"
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

Digits4 extract_digits4(const int num) {
    return (Digits4){
        .thousands = num / 1000,
        .hundreds   = num / 100 % 10,
        .tens       = num / 10 % 10,
        .units      = num % 10
    };
}

void run_task4(void) {
    long n = 0;

    puts("\n=== Task 4: sum of the first three digits of N ===\n");

    if (!read_long("Enter a positive four-digit integer N: ", &n)) {
        puts("Invalid input.");
        return;
    }

    if (n < MIN_FOUR_DIGIT || n > MAX_FOUR_DIGIT) {
        printf(
```

```

        "Error: N must be a four-digit integer (%d-%d).\n",
        MIN_FOUR_DIGIT,
        MAX_FOUR_DIGIT
    );
    return;
}

const Digits4 d = extract_digits4((int)n);
const int sum = d.thousands + d.hundreds + d.tens;

printf("\nN = %ld\n", n);
printf("Digits: %d %d %d %d\n", d.thousands, d.hundreds, d.tens, d.units);
printf("Sum of the first three digits: %d\n", sum);
}

```

4.3. Program results

```

=== Task 4: sum of the first three digits of N ===

Enter a positive four-digit integer N: 1234

N = 1234
Digits: 1 2 3 4
Sum of the first three digits: 6

```

Fig. 9 – Task 4 results

Conclusions

During the execution of this laboratory work, practical skills were acquired in designing and implementing branched algorithms using conditional statements. The software implementation successfully met all requirements across the four tasks, including coordinate quadrant determination, age group classification, piecewise function computation, and numeric digit extraction.

APPENDIX A

Block diagram of the main program control flow algorithm

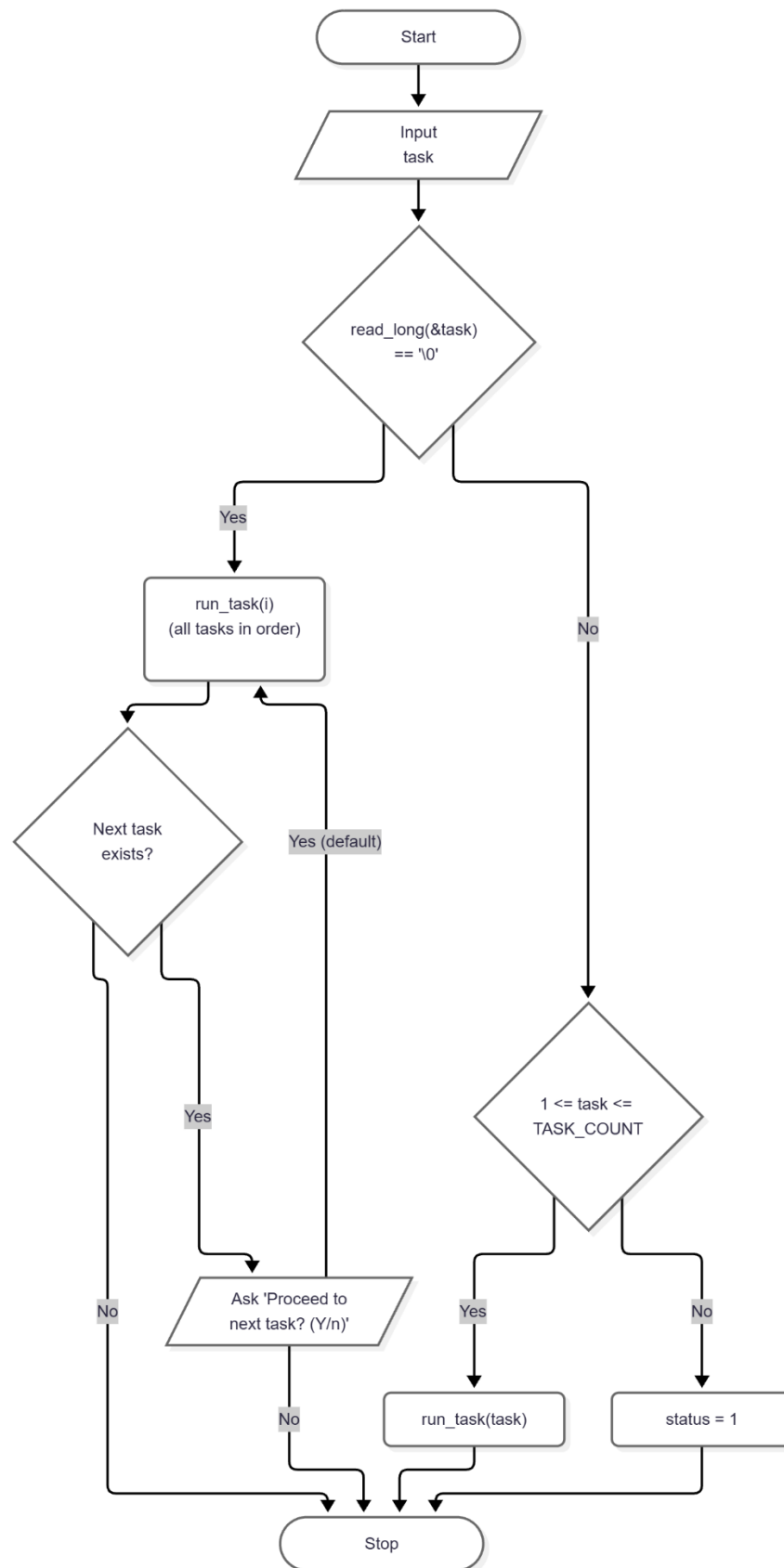


Fig. A.1 – Block diagram of the main program control flow algorithm

APPENDIX B

Source code of the “tasks.h” file

```
#ifndef TASKS_H
#define TASKS_H

#define TASK_LIST X(1) X(2) X(3) X(4)

#define X(num) void run_task##num(void);
TASK_LIST
#undef X

#define X(num) +1
enum { TASK_COUNT = 0 TASK_LIST };
#undef X

void run_task(int n);
void run_all_tasks(void);

#endif // TASKS_H
```

APPENDIX C

Source code of the “utils.h” file

```
#ifndef UTILS_H
#define UTILS_H

#include <stdbool.h>
#include <stddef.h>

bool read_double(const char *prompt, double *out_value);
bool read_doubles(const char *prompt, size_t count, ...);
bool read_long(const char *prompt, long *out_value);
bool read longs(const char *prompt, size_t count, ...);
bool confirm(const char *prompt, bool default_answer);

#endif // UTILS_H
```

APPENDIX D

Source code of the “main.c” file

```
#include "tasks.h"
#include "utils.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    printf(
        "=====\\n"
        " Laboratory work 2: Branched algorithms (variant 9)\\n"
        "=====\\n\\n"
        "Enter task number (1-%d) or press Enter to run all tasks [ALL]: ",
        TASK_COUNT
    );

    long task;
    int status = 0;

    if (!read_long("", &task)) {
        run_all_tasks();
    } else if (task >= 1 && task <= TASK_COUNT) {
        run_task((int)task);
    } else {
        printf("Error: invalid task '%ld'.\\n", task);
        status = 1;
    }

    printf("\\nPress Enter to exit...");
    getchar();

    return status;
}
```

APPENDIX E

Source code of the “runner.c” file

```
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

static void (*const TASKS[TASK_COUNT])(void) = {
#define X(num) run_task##num,
    TASK_LIST
#undef X
};

void run_task(const int n) {
    TASKS[n - 1]();
}

void run_all_tasks(void) {
    for (int i = 1; i <= TASK_COUNT; ++i) {
        run_task(i);

        if (i < TASK_COUNT) {
            char prompt[64];
            snprintf(
                prompt, sizeof(prompt), "\nProceed to task %d? (Y/n): ", i + 1
            );

            if (!confirm(prompt, true)) {
                break;
            }
        }
    }
}
```

APPENDIX F

Source code of the “utils.c” file

```
#include "utils.h"

#include <ctype.h>
#include <errno.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum { MAX_READ_VALUES = 8 };

typedef bool (*token_parser_fn)(const char *str, char **endptr, void *out);

static bool is_buffer_empty(const char *ptr) {
    while (*ptr != '\0') {
        if (!isspace((unsigned char)*ptr)) {
            return false;
        }
        ptr++;
    }

    return true;
}

static bool parse_double_token(const char *str, char **endptr, void *out) {
    errno = 0;
    *(double *)out = strtod(str, endptr);

    return *endptr != str && errno == 0;
}

static bool parse_long_token(const char *str, char **endptr, void *out) {
    errno = 0;
    *(long *)out = strtol(str, endptr, 10);

    return *endptr != str && errno == 0;
}

static bool read_line_values(
    const char *prompt,
    const size_t count,
```

```

void *const outs[],
const token_parser_fn parse
) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    if (prompt != NULL) {
        printf("%s", prompt);
    }

    char buffer[256];
    if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
        return false;
    }

    buffer[strcspn(buffer, "\r\n")] = '\0';

    const char *ptr = buffer;

    for (size_t i = 0; i < count; ++i) {
        if (outs[i] == NULL) {
            return false;
        }

        char *endptr;
        if (!parse(ptr, &endptr, outs[i])) {
            return false;
        }

        ptr = endptr;
    }

    return is_buffer_empty(ptr);
}

bool read_double(const char *prompt, double *out_value) {
    return read_doubles(prompt, 1, out_value);
}

bool read_doubles(const char *prompt, const size_t count, ...) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

```

```

void *outs[MAX_READ_VALUES];

va_list args;
va_start(args, count);

for (size_t i = 0; i < count; ++i) {
    outs[i] = va_arg(args, double *);
}

va_end(args);

return read_line_values(prompt, count, outs, parse_double_token);
}

bool read_long(const char *prompt, long *out_value) {
    return read_longs(prompt, 1, out_value);
}

bool read_longs(const char *prompt, const size_t count, ...) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    void *outs[MAX_READ_VALUES];

    va_list args;
    va_start(args, count);

    for (size_t i = 0; i < count; ++i) {
        outs[i] = va_arg(args, long *);
    }

    va_end(args);

    return read_line_values(prompt, count, outs, parse_long_token);
}

static bool equals_ignore_case(
    const char *str,
    const size_t len,
    const char *word
) {
    if (strlen(word) != len) {
        return false;
    }

```

```

for (size_t i = 0; i < len; ++i) {
    if (tolower((unsigned char)str[i]) != word[i]) {
        return false;
    }
}

return true;
}

bool confirm(const char *prompt, const bool default_answer) {
    char buffer[16];

    while (true) {
        if (prompt != NULL) {
            printf("%s", prompt);
        }

        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            return false;
        }

        const bool overflow = strchr(buffer, '\n') == NULL && !feof(stdin);
        if (overflow) {
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF)
                ;
        } else {
            const char *start = buffer;
            while (isspace((unsigned char)*start)) {
                start++;
            }

            size_t len = strlen(start);
            while (len > 0 && isspace((unsigned char)start[len - 1])) {
                len--;
            }

            if (len == 0) {
                return default_answer;
            }

            if (equals_ignore_case(start, len, "y") ||
                equals_ignore_case(start, len, "yes")) {
                return true;
            }
        }
    }
}

```

```

    }

    if (equals_ignore_case(start, len, "n") ||
        equals_ignore_case(start, len, "no")) {
        return false;
    }
}

printf("Please answer 'y' or 'n'.\n");
}
}

```